iRAYPLE Industrial Camera

SDK Development Document (LabView)



V1.1.0

Revision History

No.	Date	Version	Revision Content	Software Version
1	Aug. 2023	V1. 0	First Release.	2.4.0
2	Mar. 2024	V1.1	Added the interface of **Bool** and **String** Acquisition.	2.4.2

401674 Iraypl e 2025-01-10

Table of Contents

1 Overview 1

1.1 SDK Download 1

1.2 Compatibility Introduction 1

1.3 Installation Directory 1

1.3.1 SDK for Windows 1

1.4 Related Resources of SDK (C interface) 3

2 Calling Process and Software Introduction 5

2.1 Overall Call Process of SDK 5

2.2 Software Introduction 6

3 Sub-VI Function Introduction 9

3.1 EnumDevice.vi 9

3.2 OpenDevice.vi 9

3.3 GetIntFeatureValue.vi 10

3.4 GetDoubleFeatureValue.vi 10

3.5 GetEnumFeatureValue.vi 10

3.6 GetEnumFeatureEntryNum.vi 11

3.7 GetEnumFeatureEntrys.vi 11

3.8 GetEnumFeatureSymbol.vi 11

3.9 SetIntFeatureValue.vi 12

3.10 SetDoubleFeatureValue.vi 12

3.11 SetEnumFeatureSymbol.vi 12

3.12 IsOpen.vi 13

3.13 GetFrame.vi 13

3.14 ReleaseFrame.vi 13

3.15 ExecuteCommandFeature.vi 14

3.16 PixelConvert.vi 14

3.17 SaveImageToBmp.vi 14

3.18 StartGrabbing.vi 15

3.19 StopGrabbing.vi 15

3.20 Close.vi 15

3.21 SetBufferCount.vi 15
3.22 IsGrabbing.vi 16
3.23 GetBoolFeatureValue.vi 16
3.24 GetStringFeatureValue.vi 16
3.25 SetStringFeatureValue.vi 16
3.26 SetBoolFeatureValue.vi 17
4 FAQ (Frequently Asked Question) 18

401674 Iraypl e 2025-01-10

1 Overview

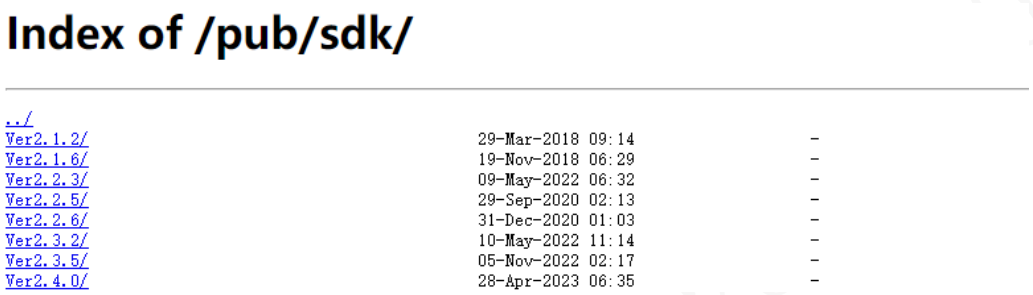
1.1 SDK Download

You can download the SDK installation package on the official website:

http://download.huaraytech.com/pub/sdk/

Please download the appropriate version of SDK.

Figure 1-1 SDK installation packages



1.2 Compatibility Introduction

Table 1-1 Operating system requirement

OS Type	Version	Note
Windows *1	Win7 SP1 32/64bit	Ensure that OS version includes the patch of SP1.
	Win8 32/64bit	
	Win10 32/64bit	
Linux (x86) *2	System Version: Ubuntu 12.04 LTS ~ Ubuntu 21.04 LTS （32/64bit）	The upgraded kernel version system is excluded.
	CentOS 6.5 ~ CentOS 32 （32/64bit）	The upgraded kernel version system is excluded.

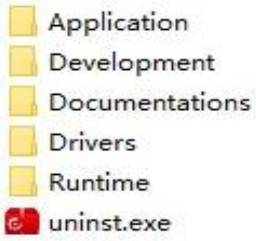
- Windows XP is no longer supported since V2.2.5 SDK.
- The supported kernel version of Linux is 2.6.32 to 5.11.0 (Inclusive).
- Use the command of **uname –a** to check the kernel version number.

1.3 Installation Directory

1.3.1 SDK for Windows

The installation directory structure of the SDK for Windows is as follows:

Figure 1-2 installation directory



In V2.1.0, user can select the installation path.

Table 1-2 Installation directory structure description

Application	● **MViewer** is compatible with 32-bit and 64-bit. ● **CamTools** is compatible with 32-bit and 64-bit.
Development	**3rdPartyPlatformAdapter** It includes two plug-in dynamic libraries of third-party algorithm platform (Halcon and Sherlock).
	DOTENT DLL It includes **.Net** library file which is imported when user create the project.
	Include It includes the header file of the SDK interface, which explains the basic properties of various interfaces. The specific usage of header files is in the **samples folder**. The other is the extension definition of the header file of the SDK, which defines the SDK error code, the user-defined parameters, the data restriction of some parameters, the enumeration of various behaviors, and the data structure of various information.
	Lib It includes the lib files.
	Samples It includes the various types of demo file, such as C#, Delphi, Lab View, MFC, Python, QT, VB and VC.

Documentations	**.Net** interface document **ActiveX** interface document **C** interface document **SDK** development document
Drivers	The installation and uninstallation software of **DShowFilter** high-performance driver. The installation and uninstallation software of **GigE** high-performance driver. The installation and uninstallation software of **PCIeCL** high-performance driver. The installation and uninstallation software of **USB** high-performance driver. The installation and uninstallation software of **CLVSP** high-performance driver.
Runtime	The compilation dynamic link library when running the SDK. It includes the third-party **GenICam** library, which is separated with two folders as 32-bit and 64-bit.

1.4 Related Resources of SDK (C interface)

Table 1-3 SDK source description

Category	Item	File Storage Folder
SDK Library	Header File	【SDK Installation Directory】 \Development\Include\IMVApi.h \Development\Include\IMVDefines.h
	Lib (32-bit)	【SDK Installation Directory】 \Development\Lib\win32\MVSDKmd.lib
	Lib (64-bit)	【SDK Installation Directory】 \Development\Lib\x64\MVSDKmd.lib
	Runtime Library (32-bit)	【SDK Installation Directory】\Runtime \Win32\MVSDKmd.dll
	Runtime Library (64-bit)	【SDK Installation Directory】\Runtime \x64\MVSDKmd.dll

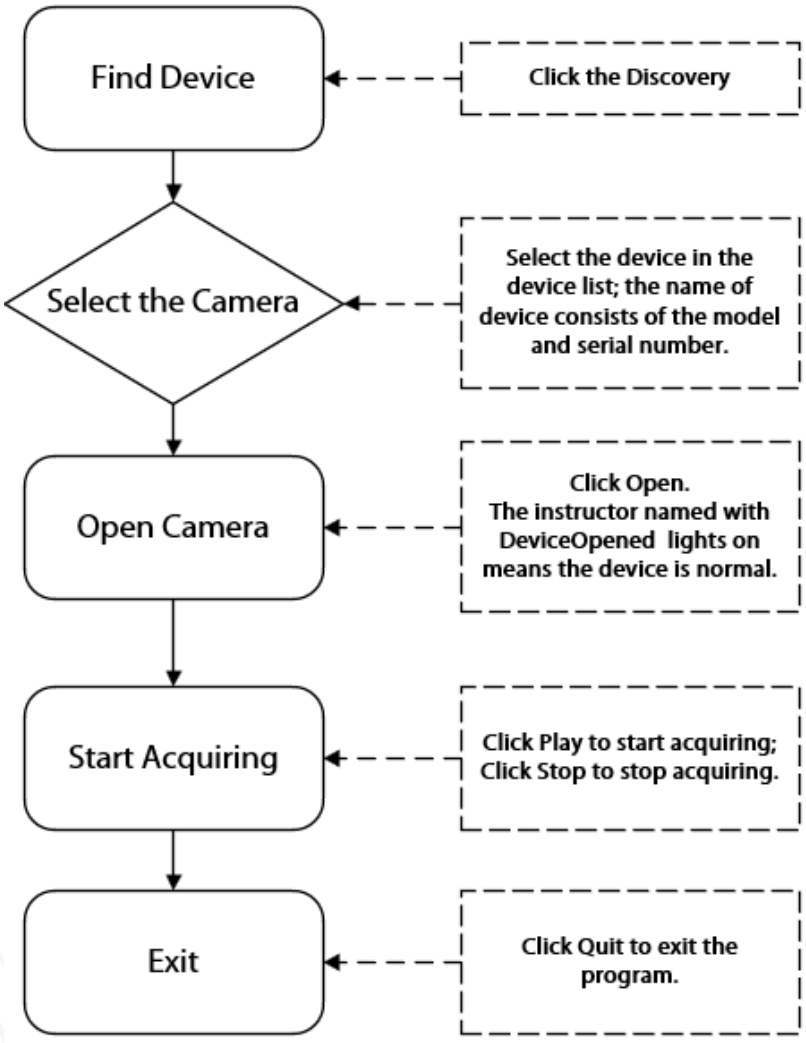
C API Document	Chinese	【SDK Installation Directory】\ Documentations\ GenICam_API_C_CHS.chm
	English	【SDK Installation Directory】\ Documentations\ GenICam_API_C_ENG.chm

401674 Irayple 2025-01-10

2 Calling Process and Software Introduction

2.1 Overall Call Process of SDK

Figure 2-1 SDK calling process



2.2 Software Introduction

Figure 2-2 LabView main interface

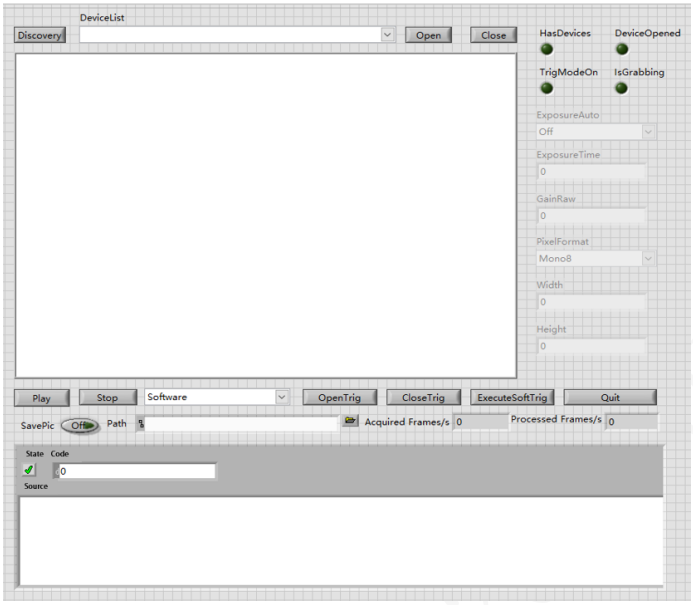
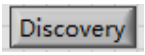
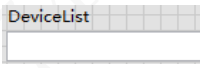
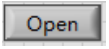
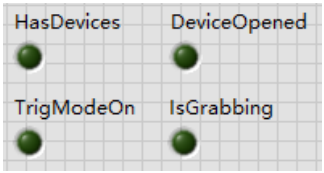
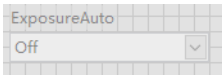
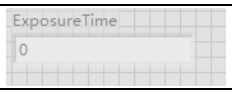
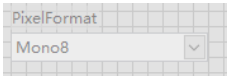
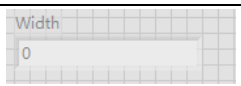
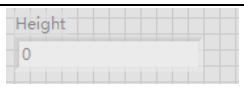
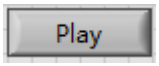
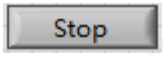
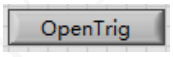
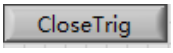
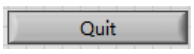
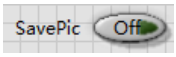
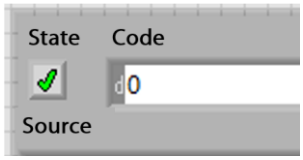


Table 2-1 LabView function description

No.	Diagram	Description
1		After clicking the **Discovery**, wait for the instructor of **HasDevice** to light up.  If LabView is in the status of Not Responding during device finding, please do not close the software.
2		Device list. The device name consists of model name and serial number.
3		Connect button. After clicking the button, the selected device will be connected, and the **DeviceOpened** light will light up.
4		Disconnect button. After disconnecting the device, the **DeviceOpened** light will go off.

5		● **HasDevice**: the instructor of device finding ● **DeviceOpened**: the instructor of device connecting ● **TrigModeOn**: the instructor of trigger enabling. ● **IsGrabbing**: the instructor of acquiring enabling.
6		The drop-down list of **ExposureAuto** includes Off, Once and Continuous.
7		For configuring the exposure time.
8		For configuring the gain value.
9		For selecting the pixel format of the device.
10		For setting the width of the image.
11		For setting the height of the image.
12		For starting the image acquisition.
13		For stopping the image acquisition.
14		For selecting the trigger mode of the device.
15		For configuring the selected trigger mode. After completing the configuration, the instructor of **TrigModeOn** will light up.
16		For disabling the Trigger mode. After disabling, the instructor of **TrigModeOn** will go off.
17		For executing the software triggering.
18		For discontinuing the operation and release the handle of device.
19		For selecting the image saving path. The image name will be the **blockID**.

20		For enabling or disable the image saving function.
21		Displays whether the frame of image is required. If it is not 0, the image is required; otherwise, the image is not required.
22		Displays the operation error information.

401674 Irayple 2025-01-10

3 Sub-VI Function Introduction

Sub-VI is the encapsulation of SDK interface, and the corresponding sub-VI can be introduced into the project to realize the corresponding functions.

- Library file path: LabView\Depend
- Executable file path: LabView\bin
- The directory of interface header files of SDK: \Development\Include\IMVApi.h
- The directory of the macro definition, enumeration, struct and callback function statement of SDK interface: \Development\Include\IMVDefines.h

3.1 EnumDevice.vi

- Description: Device enumeration.

Table 3-1 Parameter description

Parameter	Description
interfaceType [IN]	Enumeration device interface type (default type: ALL).
cameraKeyList [OUT]	Device list.
nDevNum [OUT]	The quantity of the found device.

- Function: After calling **IMV_VI_EnumDevices**, obtain the number of device and store the device information into the memory by using the **interfaceType**. Then, obtain and send the device list by calling the **IMV_VI_Assignment**.

3.2 OpenDevice.vi

- Description: Open device.

Table 3-2 Parameter description

Parameter	Description
handle [OUT]	Device handle.
pIdentifier [IN]	Specify the identifier (device key, custom device name, explicitly casting the pointer representing the IP address from char type to void type; Explicitly casting the pointer representing the index from unsigned int type to void type).
return type [OUT]	The returned value.

- Function: After calling the **IMV_VI_CreateHandle**, create device handle by adding **pIdentifier** and creating mode.



Default mode: mode 0, which means that the device handle creation is performed by indexing of the enumerated device. The returned value can be the evidence to identify whether the device is opened.

3.3 GetIntFeatureValue.vi

- Description: Obtain the attribute value of **Int**.

Table 3-3 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The obtained attribute name.
pIntValue [OUT]	The attribute value of **int**.

- Function: After calling the **IMV_VI_GetIntFeatureValue,** obtain the corresponding attribute value by adding the **handle** and **pFeatureName**.

3.4 GetDoubleFeatureValue.vi

- Description: Obtain the attribute value of **Double.**

Table 3-4 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The obtained attribute name.
pDoubleValue [OUT]	The attribute value of **Double**.

- Function: After calling the **IMV_VI_GetDoubleFeatureValue,** obtain the corresponding attribute value by adding the **handle** and **pFeatureName**.

3.5 GetEnumFeatureValue.vi

- Description: Obtain the attribute value of **Enum.**

Table 3-5 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The obtained attribute name.
pEnumValue [OUT]	The attribute value of **Enum**.

- Function: After calling the **IMV_ VI_GetEnumFeatureValue,** obtain the corresponding attribute value by adding the **handle** and **pFeatureName**.

3.6GetEnumFeatureEntryNum.vi

- Description: Obtain the number of configurable enumeration values of enumeration attribute.

Table 3-6 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The obtained attribute name.
pEntryNum [OUT]	Obtain the quantity of configurable enumeration values of enumeration attribute.

- Function: After calling the **IMV_VI_GetEnumFeatureEntryNum,** obtain the number of configurable enumeration value by adding the **handle** and **pFeatureName**

3.7GetEnumFeatureEntrys.vi

- Description: Obtain the list of configurable enumeration values of enumeration attribute.

Table 3-7 Parameter description

Parameter	Description
handle [IN]	Device handle.
EntryNum [IN]	The quantity of configurable enumeration values of enumeration attribute.
pFeatureName [IN]	The obtained attribute name.
pLbEnumEntryList [out]	The quantity of configurable enumeration values of enumeration attribute.

- Function: After calling the **IMV_VI_GetEnumFeatureEntrys,** obtain the **pLbEnumEntryList** by adding the **handle, EntryNum** and **pFeatureName**

3.8 GetEnumFeatureSymbol.vi

- Description: Obtain the **symbol value** of enumeration attribute.

Table 3-8 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The obtained attribute name.
pEnumSymbol [OUT]	The symbol value of enumeration attribute.

- Function: After calling the **IMV_VI_GetEnumFeatureSymbol,** obtain the desired symbol value of attribute value by adding the **handle** and **pFeatureName**.

3.9SetIntFeatureValue.vi

- Description: Obtain the attribute value of **Int**.

Table 3-9 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The configured attribute name.
intValue [IN]	The configured **int** value.

- Function: After calling the **IMV_VI_SetIntFeatureValue,** obtain the value of **intValue** by adding the **handle** and **pFeatureName**.

3.10 SetDoubleFeatureValue.vi

- Description: Configure the attribute value of **Double**.

Table 3-10 Parameter description

Parameter	Description
handle [IN]	The handles of devices and acquisition card.
pFeatureName [IN]	The configured attribute name.
doubleValue [IN]	The configured attribute value of **Double**

- Function: After calling the **IMV_VI_SetDoubleFeatureValue,** configure the value of **doubleValue** by adding the **handle** and **pFeatureName**.

3.11 SetEnumFeatureSymbol.vi

- Description: Obtain the enumeration attribute **symbol value**.

Table 3-11 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The obtained attribute name.
pEnumSymbol[IN]	The symbol value of enumeration attribute.
return type [OUT]	The returned value of function.

- Function: After calling the **IMV_VI_SetEnumFeatureSymbol,** configure the value of **pEnumSymbol** by adding the **handle** and **pFeatureName**.

3.12 IsOpen.vi

- Description: Determine whether the device is opened.

Table 3-12 Parameter description

Parameter	Description
handle [IN]	Device handle.
return type [OUT]	The returned value of function.

- Function: After calling the **IMV_VI_IsOpen,** determine whether the device is opened according to the returned value by adding **handle**.

3.13 GetFrame.vi

- Description: Acquire a frame of image (Synchronous acquisition mechanism for frame data).

Table 3-13 Parameter description

Parameter	Description
handle [IN]	Device handle.
pLbFrame out [OUT]	The frame data information.
timeoutMS [IN]	Timeout duration.
pData [OUT]	The starting address of memory of the image frame data.
framehandle [OUT]	Frame handle.
return type [OUT]	The returned value of function.

- Function: After calling the **IMV_VI_GetFrame,** obtain the **pData, pLbFrame out** and **framehandle** by adding the **timeoutMS** and **interface handle**.

3.14 ReleaseFrame.vi

- Description: Release the image buffer memory.

Table 3-14 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFrame [IN]	The frame data information.
pData [IN]	The starting address of memory of the fame data of image.
framehandle [IN]	Frame handle.

- Function: After calling the **IMV_VI_ReleaseFrame,** delete the **pLbFrame, interface handle, framehandle** and **pData** by adding the **ReleaseFrame.vi**.

3.15 ExecuteCommandFeature.vi

- Descriptions: Execute the command attribute.

Table 3-15 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

- Function: After calling the **IMV_VI_ExecuteCommandFeature,** execute the command attribute by adding the **handle** and **pFeatureName**.

3.16 PixelConvert.vi

- Description: Pixel format conversion.

Table 3-16 Parameter description

Parameter	Description
lbPstPixelConvertParam [IN]	Pixel format conversion parameter structure.
handle [IN]	Device handle.
pSrcData [IN]	Input image data.
pDstBuf [IN]	The applied output image data buffer.
pDstBuf out [OUT]	Output image data.

- Function: After calling the **IMV_VI_PixelConvert,** obtain the **pDstBuf** out by adding the **lbPstPixelConvertParam, Interface handle, pSrcData** and **pDstBuf**.

3.17 SaveImageToBmp.vi

- Description: Save image.

Table 3-17 Parameter description

Parameter	Description
handle [IN]	Device handle.
pLbFrame [IN]	Frame data information.
filePath [IN]	The inputted file paths.
pDstBuf [IN]	The converted image data.
pSrcData [IN]	The unconverted image data.

- Function: After calling the **IMV_VI_SaveImageToBmp,** save image into the specified path by adding **pLbFrame, filePath, pDstBuf, pSrcData** and **handle**.

3.18 StartGrabbing.vi

- Description: Stop acquiring image.

Table 3-18 Parameter description

Parameter	Description
handle [IN]	Device handle.
return type [OUT]	The returned value.

- Function: After calling the **IMV_VI_StopGrabbing,** determine whether the image acquiring is normal according to the return value by adding **handle**.

3.19 StopGrabbing.vi

- Description: Stop acquiring image.

Table 3-19 Parameter description

Parameter	Description
handle [IN]	Device handle.
return type [OUT]	The returned value.

- Function: After calling the **IMV_VI_StopGrabbing,** determine whether the image acquiring is normal according to the returned value by adding **handle**.

3.20 Close.vi

- Description: Close device.

Table 3-20 Parameter description

Parameter	Description
handle [IN]	Device handle.
return type [OUT]	The returned value.

- Function: After calling the **IMV_VI_Close,** determine whether the device is closed according to the return value by adding **handle**.

3.21 SetBufferCount.vi

- Description: Configure the buffer quantity of frame data.

Table 3-21 Parameter description

Parameter	Description
handle [IN]	Device handle.
nSize [IN]	The quantity of buffer data.
return type [OUT]	The returned value of function.

- Function: After calling the **IMV_VI_SetBufferCount,** configure the buffer quantity of frame data by adding handle and **nSize;** determine whether the device is closed according to the **return type**.

3.22 IsGrabbing.vi

- Description: Determine whether the device is acquiring image.

Table 3-22 Parameter description

Parameter	Description
handle [IN]	Device handle.
return type [OUT]	The returned value of function.

- Function: After calling the **IMV_VI_IsGrabbing,** determine whether the image acquiring is normal according to the value of **return type** by adding **handle**.

3.23 GetBoolFeatureValue.vi

- Description: Obtain the attribute value of **Bool.**

Table 3-23 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The obtained attribute name.
pBoolValue [OUT]	The attribute value of **Bool**.

- Function: After calling the **IMV_VI_GetBoolFeatureValue,** obtain the corresponding attribute value by adding the **handle** and **pFeatureName**.

3.24 GetStringFeatureValue.vi

- Description: Obtain the attribute value of **String.**

Table 3-24 Parameter description

Parameter	Description
handle [IN]	Device handle.
pFeatureName [IN]	The obtained attribute name.
pStringValue [OUT]	The attribute value of **String**.

- Function: After calling the **IMV_VI_GetStringFeatureValue,** obtain the corresponding attribute value by adding the **handle** and **pFeatureName**.

3.25 SetStringFeatureValue.vi

- Description: Configure the attribute value of **String**.

Table 3-25 Parameter description

Parameter	Description
handle [IN]	The handles of devices and acquisition card.
pFeatureName [IN]	The configured attribute name.
stringValue [IN]	The configured attribute value of **String**.

- Function: After calling the **IMV_VI_SetStringFeatureValue,** configure the value of **stringValue** by adding the **handle** and **pFeatureName**.

3.26 SetBoolFeatureValue.vi

- Description: Configure the attribute value of **Bool**.

Table 3-26 Parameter description

Parameter	Description
handle [IN]	The handles of devices and acquisition card.
pFeatureName [IN]	The configured attribute name.
boolValue [IN]	The configured attribute value of **String.**

- Function: After calling the **IMV_VI_SetBoolFeatureValue,** configure the value of **boolValue** by adding the **handle** and **pFeatureName**.

401674 Iraypl 2021-01-10

4 FAQ (Frequently Asked Question)



- The V2.4.0 or previous version of SDK has adopted the **C# interface**. If user needs to use the different digits of LabView program, the **C# library** in the **depend** shall be replaced with corresponding digit.
- The V2.4.1 of SDK has adopted the **C interface,** therefore, it cannot be operated in the 32-bit LabView. User shall displace the **MVSDK.dll** with the **MVSDK.dll** of the V2.4.2 or later version of SDK.



  - MVSDK.dll Path: (Installation path) \HuarayTech\MV Viewer\Runtime\x64(or Win32)

- When using the LabView routines in the V2.4.0 or previous version (C interface) of SDK, it might report the error message (1172).
  - ◇ Reason

    User did not click **Close** to close the device interface before stop running the routine. Therefore, the device handle is stilled occupied.
  - ◇ Solution

    Completely close and restart LabView, or restart the device.

401674 Iraytech 2021-01-10